

Smart Search — architecture review

Date: 2026-08-12. **Revised 2026-08-13** — §7 added (agreed four-mode direction and sequencing); §6 kept as the record of the options considered. Reviewed against the working tree, not the README. All numbers below are measured on this machine against the live Azure resources in `.env` and the 500-record `directory.db`.

0. Headline

The search feature isn't slow and wrong because of a bug in the retrieval layer. It's slow and wrong because of **three structural decisions**, each of which is individually defensible and collectively fatal:

- 1 **A frontier reasoning model is doing punctuation-triggered intent classification over a closed set of 7 labels.**
Measured: 20.7s at default reasoning effort, 2.1s at `reasoning_effort="minimal"` — *identical routing decision*, 1152 reasoning tokens burned to pick one of seven names. The same intent typed without a question mark answers in **10ms**.
- 2 **The tool set can't express the question users actually ask.** `get_org_chain` requires a UUID; users ask by name. There is no name→id step and no second tool call, so `SYSTEM_PROMPT` contains ~40 lines of prose routing named-person org questions into `find_people` instead, which can only ever return **one hop**. "Who is above Shaun Anderson, all the way up to the top?" returns his direct manager and a confident sentence.
- 3 **Every query with any criterion goes through Azure AI Search, and Search results are capped at 5.** "Who on the Compliance Team knows GDPR" → 12 people match, **5 are returned**. The cap is correct for fuzzy relevance ranking and wrong for structured filters, but the code can't tell them apart because it routes both through the same path.

None of the three is a retrieval-quality problem. Azure AI Search is fine: index is live, 500 docs, vectors populated, ~280–370ms per call. **Retrieval is the healthiest part of this stack.**

1. The actual pipeline (verified against code)

```

Frontend (App.tsx:128)
└─ GET /search?q=...&skill=...&office=...   ← q AND filters sent together
    │
    ▼ main.py:79 unified_search_route
    ▼ unified_search.py:70
        │
        └─ is_question(text)? ← regex: trailing "?" OR opens with who/what/.../is/are/does
            │
            └─ NO → find_people(query=text, **filters) [direct] ~10-390ms
                │
                └─ people.py:201
                    │
                    └─ exact-identifier short-circuit (name/email/slack) → SQL
                        │
                        └─ else if any criterion → search_client.search_people()
                            │
                            └─ _embed_query() → Azure OpenAI embeddings
                                │
                                └─ POST /docs/search (queryType:"full", RRF hybrid)
                                    │
                                    └─ else → SQL SELECT
                                        │
                                        └─ is_record_visible() filter ← the security boundary
                                            │
                                            └─ cap: 5 if Search was used, else 50 ← x the truncation bug
                                                │
                                                └─ single-exact-name enrichment (manager/delegate/direct_reports)
                                                    │
                                                    └─ ...only when `name=` was passed. Direct mode passes
                                                        │
                                                        └─ `query=`, so this NEVER fires from the UI.
                                                            │
                                                            └─ audit row + commit
                                                                │
                                                                └─ YES → _assisted(text) [assisted] 2-20s
                                                                    │
                                                                    └─ resolve_intent() → _real_resolve()
                                                                        │
                                                                        └─ 107 messages / ~5.2K tokens / 37 few-shots → gpt-5
                                                                            │
                                                                            └─ (no reasoning_effort set → default)
                                                                                │
                                                                                └─ Δ clean_filters is DISCARDED here — never passed to the tool
                                                                                    │
                                                                                    └─ execute_with_fallback() → execute_tool_call()
                                                                                        │
                                                                                        └─ same permission-filtered service fns as above
                                                                                            │
                                                                                            └─ _build_assisted() → _phrase() prose + trace

```

Permission filtering is sound. Both paths converge on the same `find_people/get_person/get_org_chain` service functions; `is_record_visible` and `visible_fields` run after retrieval, before response construction; Azure Search never sees the caller. The restricted-record test case (t1-15) confirms the restricted employee is genuinely absent from the response body. **I found no path where a caller receives data they shouldn't.** One latent hole is noted in §5.

Where the README is wrong

README claim	Reality
L208 "Every write to an indexed field (skills, bio, projects, title) re-indexes."	<code>update_own_bio</code> (people.py:481) is the only write path in the app. It writes <code>bio</code> — which is in <code>build_profile_text()</code> — and does not re-index. Nothing calls <code>build_search_index.py</code> on write. Not implemented.
Semantic reranking configured	The <code>semantic</code> config exists in <code>search_index_schema.json</code> and on the live index, but <code>search_client.py:158</code> sends <code>queryType: "full"</code> with no <code>semanticConfiguration</code> . It is never used at query time .
"Degrades to keyword + fuzzy without Azure OpenAI creds"	True and working, but irrelevant now — all three Azure resources are live in <code>.env</code> , so <code>_mode()</code> returns <code>"real"</code> and the mock resolver is only reachable on an <code>OpenAIError</code> . The deterministic router you already wrote is dead code in production.

2. Three failing examples, run against the current code

Example 1 — the same question costs 10ms or 8.4s depending on punctuation

```

q="Sean Wilson"           → mode=direct  10ms  1 result
q="who does Sean Wilson report to?" → mode=assisted 8375ms 1 result
                                tool=find_people({"name":"Sean Wilson"})
                                answer="Sean Wilson reports to Min-jun Sanchez."

```

Both are correct. The routing is correct — `find_people(name=...)` is what `SYSTEM_PROMPT` asks for. But 8.4 of those 8.4 seconds are the model call (Azure Search: 0ms — it short-circuited on exact name match). And note the *result card* is Sean Wilson, not Min-jun Sanchez: the answer is in the prose, the UI shows the wrong person.

Worse: in *direct* mode the identical query returns a card with **no manager field at all** —

`['availability_status', 'full_name', 'id', 'job_title', 'office', 'org_unit', 'preferred_name']`. The single-match enrichment in `people.py:385` requires `name=`, and `unified_search.py:73` passes `query=`. So the cheap path structurally cannot answer a relationship question, which is exactly why the expensive path has to exist.

Example 2 — multi-hop org traversal returns one hop and says nothing about it

```
q="Who is above Shaun Anderson, all the way up to the top?"
→ mode=assisted, 11241ms (11206ms = model)
→ tool=find_people({"name":"Shaun Anderson"})
→ answer: "Shaun Anderson reports to Michelle Dvorak."
→ results: [Shaun Anderson]
```

Ground truth (`golden_set.py:SHAUN_ANDERSON_UP_CHAIN`) is **7 people**. One was returned.

This is not a routing failure. `get_org_chain` takes `person_id: str` (a UUID) and the model has no way to obtain Shaun Anderson's UUID — there is no resolver tool and no second turn. The system prompt is explicit about the workaround:

"A NAMED person's manager question ... has no id to walk the chain with — use `find_people(name=X)` as described above instead"

`golden_set.py:190` already admits it: *"Structurally can't reach recall@3=1.0 here."* The golden set was edited to match the limitation rather than the limitation being fixed. Same pattern at `t1-06` (Katherine Byrne, 4 expected, 1 returned).

Example 3 — structured filters silently truncate to 5, and drop entirely on questions

```
filters={org_unit:"Cloud Operations Team"}      → 5 results    (9 people on the team)
filters={org_unit:"Compliance Team", skill:"GDPR"} → 5 results    (12 people match)
filters={} (no criteria → SQL path)              → 50 results
```

Filtering *more* narrowly returns *fewer* than browsing with no filter at all, because `has_criteria` sends any criterion through Azure Search, and `used_search=True` selects `MAX_SEARCH_RESULTS = 5` instead of `MAX_RESULTS = 50` (`people.py:368`). A filter-only query sends `search: "*"` with no vector — there is **no ranking at all** — and then gets the relevance cap applied to it anyway.

And in assisted mode the UI's filters are dropped on the floor:

```
q="who knows Terraform?" + office="Bangalore"
→ mode=assisted, 5733ms
→ tool=find_people({"skill":"Terraform"})      ← no office
→ Luca Horvat, Giulia Iyer, Niamh Kang, Seo-yeon Adeyemi, Emma Jung
```

`_assisted()` (`unified_search.py:93`) takes only `text`. `clean_filters` is computed at line 68 and never reaches it. **Yes — this is a frontend/backend contract mismatch contributing to "bad results."** The UI shows the filter chips as active while the backend ignores them.

Related, same category: the Filters panel is **free-text inputs** (`Filters.tsx:62,67`), not dropdowns, but the Search path uses OData `eq` (exact, case-sensitive):

```
office="Bangalore Office" → 5 results
office="bangalore"       → 0 results
```

The SQL fallback used `ilike '%...%'`. Moving retrieval to Search silently regressed this and nothing caught it.

3. Root causes, ranked

RC1 — Latency is one unset parameter, not a pipeline problem. (`tool_calling.py:489`)

configuration	latency	reasoning tokens	decision
37 few-shots, default effort	20675 ms	1152	<code>find_people({"name": "Shaun Anderson"})</code>
37 few-shots, <code>reasoning_effort="minimal"</code>	2149ms	0	identical
no few-shots, <code>reasoning_effort="minimal"</code>	2070ms	0	identical

The few-shots are **not** the problem — the prompt prefix is cached (3328/3419 tokens cached on the first call) and removing all 37 saves 79ms. gpt-5's reasoning tokens are the entire cost. It is spending a thousand tokens of deliberation to choose between seven function names.

RC2 — The tool schema is the wrong shape for org questions. No name→id resolution, no second tool call.

`execute_tool_call` runs exactly one call and returns. This is a missing capability that no amount of prompt engineering fixes — and the ~40 lines of `SYSTEM_PROMPT` prose defending the workaround are themselves a cost: they're prompt surface that has to stay consistent with the few-shots, and every routing edge case gets patched by adding more prose.

Note: `tests/test_agentic_loop.py` imports `run_turn` from `app.tool_calling`, which doesn't exist. Multi-turn work was started and abandoned. **This import error means `pytest` fails at collection and the entire suite — including the field-visibility tests marked "Done" — has been running zero tests.** With that file excluded, 58 tests pass.

RC3 — "Ranked" and "filtered" are conflated at the retrieval boundary. A relevance cap of 5 is right for "someone good with dashboards". It is wrong for "everyone on the Compliance Team with GDPR", which has an exact, enumerable, 12-person answer. The code can't distinguish them because `has_criteria` funnels both into `search_people()`.

RC4 — Model-emitted filter values are never validated against the real vocabulary. `t2-07`: the model emitted `org_unit: "Cloud Infrastructure"`. The actual department is "Infrastructure". `_org_unit_and_descendant_ids` returns `None` → hard empty, 0 results, no fallback (`execute_with_fallback` only broadens `skill` and `language`). The model is being asked to guess exact string values from a 60-unit vocabulary it has never been shown.

RC5 — `is_question()` is a punctuation coin-flip deciding a 800× cost difference. A trailing `?` routes to a 2–20s model call; its absence routes to a 10ms lookup. The two paths also have *different capabilities* (enrichment on one, filters on the other), so the punctuation changes what the user can learn, not just how fast.

RC6 — The golden eval can't measure any of this. `score()` computes `recall@3` against ground-truth sets of size 9–15, so the maximum achievable score is $3/14 = 0.21$. `t1-04` returned **all 14 correct direct reports** and scored `recall=0.21`. The five `find_mentor` questions all score exactly $0.60 = 3/5$ against dynamic ground truth that is *by construction identical* to the output. The tier-2 average of 0.584 is mostly metric artifact. **You cannot currently tell a real regression from a metric artifact**, which is a problem for a deliverable that has to demonstrate quality to Quadrant.

4. Which of your stated design decisions I think are wrong

Decision	Verdict
Azure AI Search for hybrid retrieval	Keep. Working, fast, correctly isolated from permissions. Just stop routing pure structured filters through it.

Decision	Verdict
LLM emits typed function calls, never SQL	Keep — this is the best decision in the codebase. It's what makes the permission property provable. Don't touch it.
Permission filtering in Python between retrieve and respond	Keep. Sound, tested, and the non-negotiable security property holds.
Redact-not-reject, deny-by-default	Keep.
Degrade gracefully without Azure OpenAI creds	Keep, but invert it. You built a deterministic router (<code>_mock_resolve</code>) as the <i>degraded</i> path. It's ~10ms, deterministic, unit-testable, and handles the same intents. It should be the <i>primary</i> path.
LLM routes freeform queries at all	This is the one I'd change. The intent space is 7 closed labels over a 500-person directory with a knowable vocabulary. That is a classification problem, not a reasoning problem.
Every write re-indexes	Not implemented, and I'd descope it. One write path exists (<code>bio</code>). Re-index on bio-save is ~20 lines, or defer to a nightly rebuild and say so. Don't leave the README claiming it.
Synthetic 500-record dataset	Fixed, and genuinely fine. At 500 records almost everything can be exact.

5. Bugs to fix regardless of which direction you pick

- `tests/test_agentic_loop.py` breaks collection — the whole suite runs 0 tests. Delete or stub it. (*highest priority; it's masking everything else*)
- `_assisted()` drops `clean_filters`. UI filter chips are ignored on any question.
- `MAX_SEARCH_RESULTS=5` applied to unranked filter-only queries. Only apply the tight cap when text was actually ranked (`effective_query` present).
- Office/org_unit filters are case- and exactness-sensitive via OData `eq`, behind free-text UI inputs. Either normalize server-side or make the inputs dropdowns fed by the real vocabulary.
- `app/agent_pipeline.py` is broken dead code — imports `app.models.task` (`OnboardingTask`), which does not exist. It's a policy-RAG file from a different template. Delete it before it's discovered in a review.
- Latent security hole:** `manager/delegate` `PersonRefs` are attached without an `is_record_visible` check on the referenced person (`people.py:411`, `org_chart.py:136`, `_build_detail`). Currently unexploitable — neither of the 2 restricted employees manages anyone or is anyone's delegate — but it's one seed change away from leaking a restricted person's name to a non-HR caller. Comments call it intentional; I'd add the check.
- `execute_tool_call` does `args.setdefault("depth", 10)` — if the model omits `depth` on an "up" call, `_phrase()` reports the CEO as the answer to "who is my manager".
- README §208 and the semantic-rerank claim don't match the code.

6. Architecture options

Today is Aug 12. Deadline Aug 20–21 → ~8 working days. All effort estimates assume you, solo, also carrying the frontend.

Outcome: the direction taken is Option 1 + Option 2 + Option 3, restructured into the four-mode design in §7. Option 4 (agentic loop) is explicitly descope. This section stays as the record of what was weighed.

Option 1 — Patch in place

~1 day. Fix \$5 items 1–4, set `reasoning_effort="minimal"`.

- **Solves:** latency 20.7s → ~2.1s. Filter truncation. Dropped filters. Case sensitivity. Test suite runs again.
- **Does not solve:** multi-hop org (RC2), hallucinated filter values (RC4), the 2s floor on any question, the punctuation coin-flip (RC5), the broken eval (RC6).
- **Cost:** almost none. Nothing structural changes.
- **Deadline:** trivially fits. **Do this regardless of what else you choose** — it is a strict subset of every other option.
- **Risk:** you demo a system where "who is above X all the way to the top" is confidently wrong.

Option 2 — Re-split retrieval: SQL owns filters, Search owns fuzzy + semantic

+1–2 days on top of Option 1.

Change `has_criteria` so Search is invoked only when there's **text to rank** (`effective_query`) or when the request is genuinely semantic. Pure structured filters (`skill + org_unit + office + level + language + available`) go to SQL: exact, uncapped (to 50), case-insensitive, hierarchy-aware, ~10ms, no embedding call.

- **Solves:** RC3 completely and RC4 partially (SQL can `ilike`-match a near-miss org unit where OData `eq` can't). Removes an embedding round-trip and a Search round-trip from every filter query. Filter queries drop to ~10ms.
- **Costs:** you lose relevance *ordering* on filter results — but there was none (`search:"*"`). You give up the story "everything goes through hybrid search," which may matter to how you present this to Quadrant. Counter-story: "we use the right index for the right query shape" is a better engineering story anyway.
- **Deadline:** comfortable.
- **Honest tradeoff:** at 500 records this is clearly right. At 50,000 you'd want the filters back in Search for pagination and faceting. Say so explicitly rather than pretending it scales.

Option 3 — Invert the routing: deterministic first, LLM as fallback (*my recommendation*)

+2–3 days on top of Options 1+2. Total ~4–5 days, leaving 3–4 days of buffer.

Promote `_mock_resolve` from degraded-mode fallback to primary router, and give it the two things it's missing:

- 1 **A name resolver.** 500 names loaded in-process; exact match → fuzzy (rapidfuzz) → candidate list. This is the missing name→id step. `get_org_chain` becomes reachable for named people, which **fixes multi-hop org traversal properly** (RC2) — no second model call, no second turn.
- 2 **A vocabulary snapper.** Org units, skills, offices, languages come from the DB. Any filter value — from the user, the UI, or the model — is snapped to the nearest real value or reported as unresolvable. **Fixes RC4** and makes the free-text filter inputs work.

The LLM fires only when the deterministic router has **no confident match** — genuinely novel phrasings — and its output then goes through the same vocabulary snapper. `is_question()` stops being a cost switch and becomes purely a presentation switch (do we render an overview?).

- **Solves:** RC1, RC2, RC3, RC4, RC5. Typical query drops to ~10–40ms. Routing becomes unit-testable, so the golden set can be run offline in seconds instead of at 8s/question against a rate-limited endpoint.
- **Costs:**
- You write and maintain routing rules. Genuinely more code than a prompt.
- Coverage gaps are silent — a phrasing you didn't anticipate falls through to the LLM (slow but correct), or to `find_people(query=...)` (fast but possibly off-target).

- **The "we use AI for routing" story weakens.** Be ready to say: the LLM is the fallback and the phrasing layer; the routing is deterministic *because* that's what makes permissions and latency auditable. For an internal directory at Quadrant, that's the stronger pitch — but it is a real change in how the project presents.
- **Deadline:** fits with buffer, and each piece (resolver, snapper, router promotion) is independently shippable — if you run out of time after the resolver, you've still fixed multi-hop.

Option 4 — Agentic multi-turn loop

4–6 days, high variance. I'd not do this before Aug 20.

Let the model plan 2–3 calls (`find_people(name=X) → take id → get_org_chain(id, up, 10)`), with a `run_turn` loop feeding results back. This is clearly what `tests/test_agentic_loop.py` was reaching for.

- **Solves:** RC2 in the most general way — arbitrary compositional questions ("who on Priya's team knows Terraform and is free this week") become answerable.
- **Costs:** 2–3× the model calls per query → **6s best case** with `reasoning_effort="minimal"`, 20–60s at current settings. Non-determinism compounds across turns. Every turn is a new place for the model to emit an unvalidated argument. The audit trail gets harder to reason about, and you'd want to re-verify the permission property per turn.
- **Deadline:** the tail risk is a demo that intermittently takes 30s. **Not worth it in 8 days.**
- ****When it is right:**** post-deadline, on top of Option 3 — deterministic router handles the 90% of traffic that's one hop, the agentic loop handles the genuinely compositional tail.

Cross-cutting: fix the eval before you trust any of this

~half a day, do it first. Replace `recall@3` with `recall@k` where $k = |\text{ground truth}|$, or plain set precision/recall. Split "did it route correctly" (exact tool+args match, deterministic, no model needed for the covered cases) from "did retrieval return the right people". Right now you cannot measure whether any of the above helped, and `t1-04` scoring 0.21 while returning a perfect answer will actively mislead you under deadline pressure.

7. Agreed direction — four modes

Replace the seven topic-shaped tools with **four modes split by retrieval strategy**. This is the right axis: strategy is what determines cost, latency, and failure shape, whereas "topic" (which is how the current seven tools are organised) determines none of them.

#	Mode	Answers	Retrieval	Model calls	Latency
1	Structured lookup	"Who works in Payroll?" · "Compliance Team, GDPR, expert level"	Constrained query object → policy layer → SQL	1 (routing)	~10ms + routing
2	Rule-ranked	"Who can teach me Terraform?"	<code>find_mentor()</code> → SQL, multi-factor sort	1 (routing)	~10ms + routing
3	Semantic	"I'm stuck on X, who can help?"	Project vectors → cosine → project→employee hop	2 (routing + summary)	~2–3s
4	Org traversal	"Who does X report to?" · "Everyone above X"	Name resolver → <code>get_org_chain</code> recursive CTE	1 (routing)	~10ms + routing

Mode 4 is the one that was missing from the first draft of this design and it is the highest priority. It fixes RC2 — the biggest measured correctness failure in §2 (1 of 7 people returned, 11.2 seconds). A flat filter object cannot express a recursive CTE, so mode 1 can't absorb it. The missing piece is a name resolver: 500 names in-process, exact → fuzzy, "Shaun Anderson" → UUID, then walk the existing cycle-guarded CTE in `org_chart.py`. No model call, no second turn.

Mode 2 is already done. `find_mentor` works correctly today — its 0.60 eval score was pure metric artifact (§ RC6); the ranking matched dynamic ground truth exactly. Keep as-is.

Mode 1: the constrained query object

The model emits a validated Pydantic object, never SQL. Every dangerous slot is an enum:

```
Field = Literal["full_name", "job_title", "org_unit", "office", "work_email",
               "skills", "cost_centre", ...]      # the 21 real fields
Op     = Literal["eq", "in", "contains"]

class Filter(BaseModel):
    field: Field; op: Op; value: str | list[str] | bool

class PeopleQuery(BaseModel):
    select: list[Field]; filters: list[Filter]
    order_by: Field | None = None
    limit: int | None = None
```

The policy layer operates on the **object**, before any SQL exists — so enforcement stays a set intersection, not a parser:

```
def enforce(q: PeopleQuery, caller) -> PeopleQuery:
    allowed = ALLOWED[caller.role]                # existing permissions.py
    q.select = [f for f in q.select if f in allowed] # redact, don't reject
    q.filters = [f for f in q.filters if f.field in allowed] # ← filtering ON a field leaks it
    if caller.role != "hr":
        q.filters.append(Filter(field="availability_status", op="ne", value="restricted"))
    q.limit = min(q.limit or 20, MAX_RESULTS)      # model may hint, never widen
    return q
```

That second filter line matters: `WHERE cost_centre = 'X'` reveals membership through which rows come back, without ever projecting the column.

`limit` is where the §RC3 truncation fix lands — the model hints at how many the question wants (1 for "who's my boss", all 12 for "who knows GDPR"), clamped server-side. Note most of this is deterministic from query shape anyway: exact identifier → 1, structured filter → all matching, fuzzy text → top-k.

Explicitly rejected: model-generated SQL with a rewriting policy layer. It converts a provable set-membership check into an unbounded parsing problem (aliases, expressions, subqueries, CTEs, `UNION`), and aggregate/`HAVING`/`COUNT` inference attacks survive even a correct row-and-column rewriter. It would also make latency worse (longer output) and deployment riskier (the SQLite/T-SQL divergence already documented at `org_chart.py:52–64`).

Mode 3: prerequisites and where the vectors live

Blocker — the fuel doesn't exist yet. 114 of 118 project descriptions are the literal template `f"{name} – owned by {dept}."` (`seed.py:904,950`), median length 58 characters:

```
'Infrastructure Tooling Upgrade – owned by Infrastructure.'
'Infrastructure Workflow Automation – owned by Infrastructure.'
```

There is no semantic content to embed — this would be name-matching with an embedding round-trip bolted on. Fix in `seed.py`, not in the architecture: write real 2–4 sentence descriptions (problem, tech, what went wrong) for 118 projects. Generate offline once, paste in. Half a day, and it is what makes mode 3 work at all.

Vectors live in our own DB, not in an Azure AI Search index. Measured on the live service:

```
indexes on this service: employees-index, group-1, laidbackhr-knowledge-v1
indexesCount: usage=3 quota=15
```

The "one index per team" rule is an internal allocation on a shared Basic-tier service, not a platform limit — 12 slots are free. But at this corpus size an index is the wrong tool regardless:

```
118 projects × 1536 dims × 4 bytes = 725 KB
brute-force cosine over 118 vectors ≈ sub-millisecond with numpy
```

A `project_embeddings` table (`project_id`, `vector` BLOB), loaded once at startup. Keyword half is `name/description ILIKE` in SQL; fuse the two rankings with ~15 lines of reciprocal rank fusion. This consumes no allocation, behaves identically on SQLite and Azure SQL, and makes re-embedding on description change one API call and one row update. Revisit if the corpus ever reaches ~10k projects — the retrieval interface stays the same, only the implementation moves.

Three things to get right:

- **Confidential projects are never embedded.** Same structural exclusion `build_profile_text` already applies (`search_index.py:88`). 2 projects. Excluding at index time beats filtering at query time — it makes "who's connected to Project Nightingale" impossible rather than merely handled.
- **The project→employee hop needs ranking.** Median 9 members per project, max 52. A good semantic match on a 52-member project otherwise returns 52 people. Weight by current-vs-past membership, owner/lead vs contributor, and recency.
- **The summarisation call sits inside the trust boundary.** It sees real employee data, so permission filtering must run *before* it, never after.

Routing between the modes

Deterministic router first (promote `_mock_resolve`, currently dead code in production), LLM as fallback for phrasings it doesn't cover — with `reasoning_effort="minimal"` set either way (20.7s → 2.1s, §RC1). All model-emitted filter values pass through a vocabulary snapper built from the DB's real org units, skills, offices and languages, which fixes the hallucinated "`Cloud Infrastructure`" hard-empty (§RC4).

Four distinct modes are far more tractable to route than seven topic tools, and misroutes now degrade **softly**: "who can help with Terraform" landing in mode 1 returns people with the Terraform skill — a decent answer. Today the same misroute returns five fuzzy name matches. Mode 3 is the natural default for anything descriptive the router can't place.

8. Sequencing

Revised for Aug 13 → deadline Aug 20–21: **~7 working days**, solo, also carrying the frontend.

Day	Work
1 (½)	Fix the eval metric (§RC6). Delete/stub <code>test_agentic_loop.py</code> , delete <code>agent_pipeline.py</code> . Baseline run.
1 (½)	Quick wins: <code>reasoning_effort="minimal"</code> , pass filters into <code>_assisted</code> , cap fix, case-insensitive filters.
2	Mode 4 — name resolver → <code>get_org_chain</code> . Highest-value single change.
3–4	Mode 1 — <code>PeopleQuery</code> + <code>enforce()</code> + SQL compiler. Vocabulary snapper. Promote deterministic router.
5	Real project descriptions in <code>seed.py</code> . Fix the manager/delegate visibility check (§5.6). Re-index on bio write, or descope it and correct the README.
6–7	Mode 3 — embed projects, cosine + RRF, hop ranking, summarisation call.

Day	Work
8–9	Deploy, buffer, demo rehearsal.

Modes 1, 2 and 4 are what make the system **correct** — they fix every failure measured in §2. Mode 3 is what makes it **impressive**. That ordering is deliberate: mode 3 is built last, on top of a system that already works, where it can slip without sinking the demo.

Re-run the (fixed) eval after each of days 1, 2, 4 and 7 so every change has a measured before and after.

Descoped

- **Agentic multi-turn loop** (§6 Option 4) — 2–3× model calls per query, non-determinism compounding across turns, and a demo that intermittently takes 30s. Right layer to add *after* the deadline, on top of mode 4.
- **Model-generated SQL** — see §7 mode 1.
- **Moving structured filters into Azure AI Search** — reverted by mode 1; Search keeps fuzzy-name and semantic work only.